Application Proxy (WAF) with rule AI-generated

Tirocinio CyberChallenge

Abstract

Questo progetto presenta una soluzione innovativa per migliorare la capacità difensiva dei team partecipanti alle competizioni CTF Attack&Defence, dove rapidità e precisione nella mitigazione degli attacchi rappresentano fattori decisivi. Sebbene il patching del codice rimanga uno dei metodi più efficaci per neutralizzare le vulnerabilità, esso comporta rischi elevati e richiede tempo, rendendo necessario l'impiego di meccanismi complementari come i WAF. Tuttavia, la progettazione manuale di filtri affidabili è complessa, soggetta a errore e difficilmente adattabile al ritmo dinamico della partita.

La soluzione proposta introduce un WAF dinamico, dotato di un wrapper intelligente e collegato a un ambiente simulato in cui viene replicato il traffico reale. Agenti AI monitorano il comportamento del sistema, identificano pattern malevoli, generano o aggiornano le regole del WAF e ne verificano l'efficacia prima dell'applicazione. Questo approccio consente di testare in sicurezza l'impatto di nuove regole, evitare conflitti o falsi positivi e garantire che la protezione rimanga coerente con lo SLA del servizio.

L'architettura è composta da due componenti principali: una VM reale che ospita i servizi e il WAF, e una VM simulata che replica l'ambiente di gara, riceve un clone del traffico e fornisce agli agenti AI uno spazio isolato per analisi e validazioni. Il risultato è un sistema capace di adattarsi rapidamente agli attacchi, creare filtri sempre più efficaci e ridurre il carico operativo sul team, migliorando la resilienza complessiva durante la competizione.

Introduzione

Nelle CTF Attack&Defence, come quelle affrontate durante la nazionale, emerge in modo evidente quanto sia fondamentale disporre di un'infrastruttura capace di sostenere sia l'attacco sia la difesa in maniera efficace. Sul fronte difensivo, una delle attività più centrali è il *patching*: la modifica del codice sorgente del servizio per eliminare una vulnerabilità o neutralizzare un attacco individuato.

Il team parte dall'analisi degli attacchi, sfruttando strumenti come Caronte o Tulip per osservare chiaramente l'exploit utilizzato e produrre una patch mirata. Questa tecnica rappresenta la difesa più completa e robusta, ma porta con sé rischi significativi: un patching impreciso può introdurre nuove vulnerabilità, generare malfunzionamenti o causare errori SLA dovuti alla temporanea incoerenza con il game system (come problemi nel database).

Per ridurre tali rischi, i team che nelle scorse edizioni hanno raggiunto le prime posizioni non si affidano esclusivamente alle patch. Integrano invece l'uso di WAF (Web Application Firewall), che permettono di filtrare le richieste in base ai loro attributi o al contenuto del payload, bloccando pattern di attacco noti. Questo approccio consente una risposta immediata, evita downtime del servizio (se i filtri sono progettati con attenzione) e mantiene intatta la coerenza del database.

L'adozione di un WAF, tuttavia, introduce nuove sfide operative, tra cui:

- Riconoscere gli attacchi e individuare pattern affidabili;
- Progettare filtri realmente efficaci nella mitigazione delle minacce;
- Garantire che i filtri non interferiscono con il normale funzionamento del servizio.

Queste difficoltà rendono evidente la necessità di un approccio più rapido, flessibile e affidabile.

Soluzione

La soluzione individuata consiste nella realizzazione di un WAF capace di generare filtri in modo rapido e soprattutto dinamico, così da adattarsi all'evoluzione della partita. Perché queste regole siano realmente affidabili, è però necessario disporre di un ambiente sicuro in cui poterle testare e validare, valutando l'impatto che hanno sulla virtual machine e anticipando eventuali effetti collaterali sul game system.

Poiché le regole devono convivere in un ambiente condiviso, è inoltre fondamentale verificare come interagiscono tra loro, se introducono conflitti o causano duplicazioni indesiderate. Un WAF progettato in questo modo consentirebbe di ottenere una difesa adattiva, ridurre i tempi di reazione e ottimizzare il lavoro del team durante la competizione.

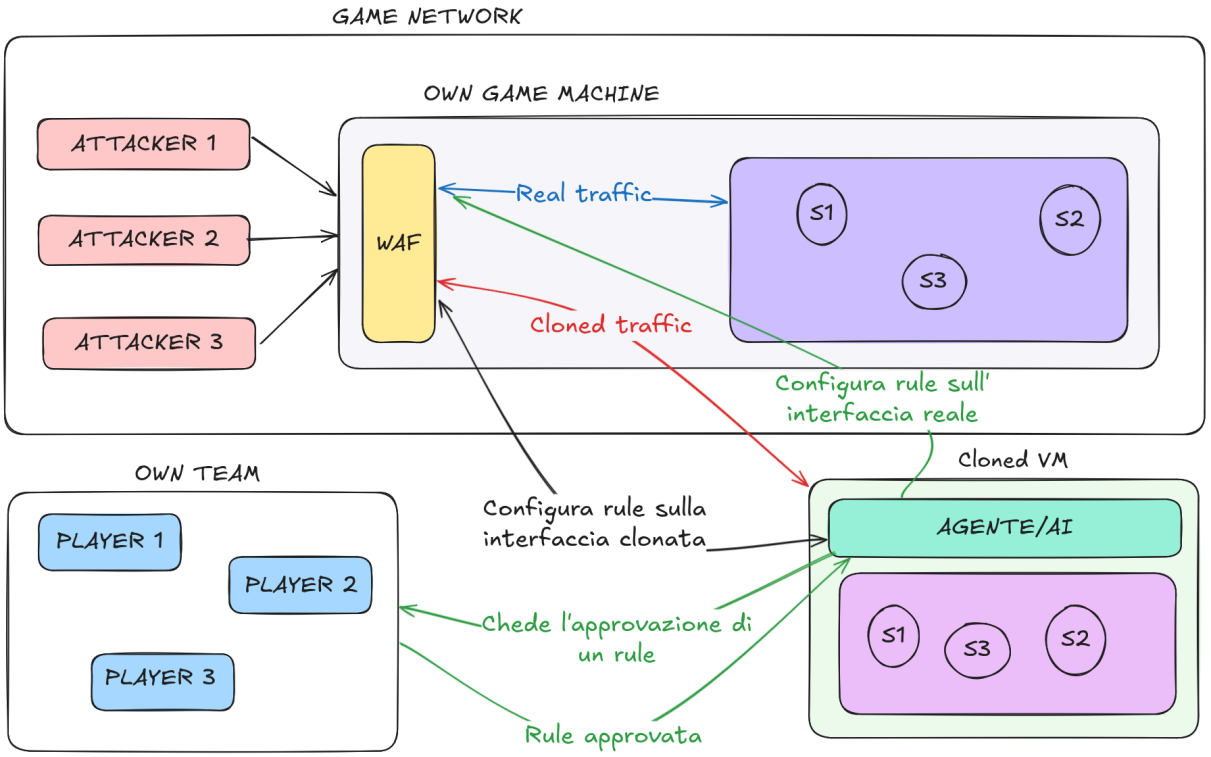
Idea

L'utilizzo di un WAF tradizionale non è sufficiente per affrontare il nostro problema: nell'era dell'automazione è necessario adottare strategie capaci di adattarsi rapidamente al contesto e di garantire un flusso operativo continuo. Un semplice modello di pattern recognition non basta; serve un sistema in grado di **analizzare dinamicamente il traffico** e di generare **regole WAF nuove o aggiornate** tramite agenti AI.
Queste regole devono essere testate in un **ambiente simulato**, dove il traffico reale viene duplicato. Questo permette di valutare l'efficacia e l'affidabilità delle regole prima della loro applicazione nel sistema reale.

Il sistema proposto integra quindi tre componenti fondamentali:

1. **Un WAF esistente**, preso da progetti già consolidati (come Firegex o Suricata).
2. **Un ambiente simulato**, in cui viene inviato un clone del traffico reale.
3. **Agenti AI**, che osservano il comportamento del sistema simulato e propongono o generano automaticamente nuove regole per mitigare il traffico malevolo.

Non è previsto lo sviluppo di un WAF da zero: l'obiettivo è realizzare un **wrapper** che aggiunga funzionalità dinamiche e intelligenti a strumenti già esistenti.



Schema Astratto del sistema

Funzionamento

L’architettura divide l’applicativo in due parti principali:

VM Reale

Questa VM rappresenta il punto centrale del sistema, dove risiedono i servizi reali e il WAF. Qui:

- arriva il traffico reale degli attaccanti;
- il WAF applica le regole attive;
- lo stesso traffico viene duplicato e inoltrato all’ambiente simulato.

Questa separazione evita che la simulazione impatti sulle risorse della VM reale (che ad esempio può avere limiti rigidi come 4 GB di RAM).

VM Simulato

Questa macchina ospita:

- i servizi simulati;
- il clone del traffico reale;
- gli agenti AI che osservano, valutano e propongono nuove regole.

Ogni volta che un giocatore deve creare una nuova regola, il test viene eseguito prima nell’ambiente simulato, che riproduce fedelmente il traffico della partita in corso.

Durante la simulazione:

- un agente monitora l’impatto della regola sullo SLA del servizio;
- se il livello di servizio scende o se la mitigazione non è sufficiente, l’agente modifica la regola o suggerisce correzioni;
- una volta verificata l’efficacia, la regola approvata viene riportata nel WAF reale.

Obbiettivi

Fase 1

- [ ] WAF Setup con wrapper per le rule user friendly
- [ ] Creazione dell’ambiente simulato in modo automatico
- [ ] Configurazione WAF con clonazione traffico

Fase 2

- [ ] User interface per interazione e gestione utente dell'agente e del WAF
- [ ] API per comunicazione con WAF e Agente (encrypted)
- [ ] Server MCP per la comunicazione con il WAF
- [ ] Agentic Tools per parser del traffico di rete (labelling) e memorization
- [ ] Agentic Tools per creazione di regole
- [ ] Suggestion delle rule

Fase 3

- [ ] Integrazione del agente con ambiente simulato e reale
- [ ] Retrieving delle metriche relative alle rule
- [ ] Modello di analisi